

Case Study — Agentic Code Auditor

Zynxis Agentic AI Internship · Week 8 Capstone · Faozan Mujtaba

The problem

LLM code review has a trust problem, not an accuracy problem. A reviewer who cannot tell which findings are proven and which are guesses must verify all of them — which costs more than reading the diff. And a model asked "is this code safe?" answers differently on consecutive runs over identical input, so nothing can be regression-tested or cited with confidence.

The approach

Separate what can be proven from what can only be judged, and label every finding accordingly.

Deterministic first. An AST analyser walks the Python syntax tree for thirteen rule classes — ``eval``, ``shell=True``, `unsafe `pickle`/`yaml``, hardcoded credentials, swallowed exceptions, mutable defaults, missing HTTP timeouts. Reasoning over structure rather than text means `"never call eval(x) here"` inside a string does not fire. A secret scanner runs alongside, pairing provider key patterns with Shannon entropy to catch unknown-provider keys.

The model second, and narrowly. Only files the deterministic stages already flagged — plus the largest few — go to the LLM, on the reasoning that a file with one confirmed problem is the likeliest place to find another. Auditing every file with a model is how a token budget gets spent describing ``__init__.py``.

Every finding is labelled. ``confirmed`` (the syntax tree proved it), ``probable`` (strong signal, worth a glance), or ``advisory`` (the model's opinion about code it could not execute). Presenting those as equals is what makes AI review untrustworthy; separating them is what makes it useful.

Making it production-shaped

Week 5 lost a report section to an unhandled HTTP 429 mid-run. That failure drove the engineering: retry with jittered backoff honouring ``Retry-After``; a circuit breaker that halts LLM review on repeated quota errors instead of grinding through doomed requests; a content-addressed disk cache so re-auditing unchanged files is free; per-run token accounting; and degradation to a complete static-only report when the model is unreachable. 18 tests cover the rules, each with positive and negative cases.

Results

Against this repository — 37 files, ~4,600 lines, all eight weeks:

	Cold run	Warm run
Findings	34 (3 critical · 5 high · 8 medium · 18 low)	34
LLM calls / tokens	7 / 18,315	0 / 0

The most useful validation was the auditor finding a genuine issue in my own Week 1 code: ``eval()`` in the ReAct agent's calculator tool. It is sandboxed — builtins stripped — but sandboxed ``eval`` is a documented escape target, and exactly what a self-review misses.

Equally instructive was what it got wrong. The first run reported 12 findings, 7 of them false positives: an enum member named ``SECRET = "secret-scan"`` read as a hardcoded credential, and its own test fixtures — including AWS's published example key — read as leaked secrets. Both were fixed by adding discrimination (a lowercase-slug and entropy filter; a test-path policy) rather than deleting rules. Suppressed findings are counted and listed in the report, because an auditor you cannot audit is not worth running.

Next steps

Languages beyond Python (tree-sitter); diff mode, so it audits a pull request's changed lines and fits in CI; CWE mapping; and a deliberate false-positive corpus, since both classes above were only caught by running the tool on itself.

What the eight weeks add up to

Each week fed this capstone: ReAct's reason-then-act loop (1), tool calling (2), state handling (3), retrieval and grounding (4), multi-stage orchestration and the rate-limit lesson (5), deterministic-core-plus-LLM-judgement (6), and validating model output before trusting it downstream (7). The through-line, and the thing I would keep: decide what the model is genuinely better at than code, give it only that, and make the boundary visible in the output.